

Ed's Skills

Two skills form the core design-to-implementation workflow for Drydock and related projects.

/thinkthrough <subject>

Purpose: Didactic, subtractive design discussion. Think through a command or topic conversationally — at your altitude — without any code edits, spec changes, or plans being produced unasked.

When to use: You want to reason through a design before building it. You're not sure what you want yet. You want a collaborator who asks one question at a time and doesn't run ahead.

What it produces: A structured notes file at `docs/notes_<subject>.md` with decisions captured automatically as they're made. Sections are tagged with:

- `spec:approved` — this decision changes a behavioral contract; needs to go into the spec
- `spec:na` — design decision, no spec change needed
- `spec:applied` — already written into `Drydock_Specification.md`
- `impl:unimplemented` — code needs to be written
- `impl:implemented` — already built

Workflow: `/thinkthrough analyze` → discuss → decisions auto-captured → Close Out compacts the session → `/apply-notes analyze` to implement.

Commands:

- `/thinkthrough <subject>` — start or continue a session
 - "close out" or `/thinkthrough close` — compact and finish the session
 - "halt" or `/thinkthrough stop` — close out and exit
-

/apply-notes <subject>

Purpose: Surface all flagged items from a notes file as a numbered selectable list, then implement the ones you choose.

When to use: After a `/thinkthrough` session, when you're ready to build. You want to see all the pending code changes and spec changes, pick which to implement, and have them done in one pass.

What it surfaces:

- **CODE CHANGES** (`impl:unimplemented`) — numbered 1, 2, 3... — these are the priority; implement by editing source files, tests, prompts, Rigging templates
- **DOC CHANGES** (`spec:approved`) — lettered A, B, C... — reconcile approved decisions into `Drydock_Specification.md`; always shows diff before writing

Selection syntax: reply with numbers, letters, ranges, or keywords:

- `all code` — implement all CODE items

- `all docs` — apply all DOC items (with diff confirmation per item)
- `all` — both
- `1 3 5 A` — specific items

After implementation: flags flip (`impl:unimplemented` → `impl:implemented`, `spec:approved` → `spec:applied`), tests run, SOUNDINGS updated, commit made.

The workflow

```
/thinkthrough <subject>  ← design conversation, decisions captured
    ↓
  close out              ← compact session to notes file
    ↓
 /apply-notes <subject>  ← select and implement
    ↓
   /clear                ← start fresh for next topic
```